

전병선

문제를 구조적으로 해결하는 Android Developer

github.com/dogdduddy · dogdduddy@gmail.com · 010-9612-2630

Kotlin 기반 상용 서비스 Cochi의 Android를 단독으로 맡아 설계와 구현, 운영 전 영역을 책임지고 있습니다. 운영 중 드러난 문제는 현상을 덮는 데서 멈추지 않습니다. CS로 접수된 채팅 끊김을 네트워크 계층의 원인까지 추적해 복구 중심으로 구조를 다시 설계했고, 개인 앱 Took에서는 사용자 행동 로그를 직접 설계해 초기 가설이 틀렸음을 데이터로 확인하고 제품 방향을 수정했습니다. 문제를 끝까지 추적하는 것, 그리고 데이터로 판단하는 것. 이 둘이 제가 일하는 방식입니다.

Work Experience

Tyrell

2025.06 - 현재

B2B2C AI 코칭 플랫폼 Cochi의 Android를 단독으로 맡아, 아키텍처 설계부터 기능 구현, 운영까지 전 영역을 주도하고 있습니다.

실시간 AI 채팅 안정성 개선 (WebSocket)

Cochi의 코칭 대화는 실시간 WebSocket으로 오가기 때문에, 연결 안정성이 곧 대화 경험의 핵심입니다.

- 문제** 상담사를 통해 "채팅 도중 코치가 더 이상 응답하지 않는다"는 사용자 문의를 전달받았습니다. 직접 재현해 보니, Wi-Fi와 셀룰러가 전환되는 순간 유지되던 WebSocket 연결이 끊어졌고, 이때 클라이언트가 제대로 재연결하거나 재동기화하지 못해 통신이 두절되고 있었습니다.
- 원인 분석** 네트워크가 전환되는 순간 단말의 IP가 바뀌고, 그 위에서 유지되던 연결이 끊어지는 것으로 확인했습니다. 네트워크가 전환되는 한 끊김 자체는 피할 수 없다고 판단해, 끊김을 막기보다 끊긴 뒤 안전하게 복구하는 쪽으로 방향을 잡았습니다.

○ 해결

끊김, 중복, 지연 세 가지 경계 상황을 각각 처리했습니다.

끊김

재연결되면 서버가 그동안 전달하지 못한 메시지를 한꺼번에 보내고, `end_of_pending` 신호로 밀린 메시지가 어디까지인지 알립니다. 클라이언트는 메시지를 받을 때마다 읽음 응답(ACK)을 보내, 서버와 클라이언트의 대화 내용이 어긋나지 않도록 했습니다.

중복

클라이언트가 메시지마다 고유 ID(Snowflake 기반)를 부여해, 같은 메시지가 다시 전송되더라도 중복임을 식별할 수 있게 했습니다.

지연·실패

전송 후 7초 안에 응답이 없으면 실패로 간주해 '전송 실패' 상태를 표시하고, 백그라운드 재연결(최대 10회) 시 자동으로 다시 보냅니다. 재시도 횟수를 제한하기보다, 다시 보내도 같은 메시지임을 식별할 수 있게 만들어 반복 전송에 대비했습니다.

● 결과

문제를 발견했던 것과 같은 방식으로 Wi-Fi와 셀룰러를 전환해 가며 끊김 상황을 직접 재현해, 메시지 누락과 중복이 없는 것을 검증했습니다. 개선 이후 동일한 증상의 CS는 접수되지 않았습니다.

AI 대화의 턴(turn) 경계 설계: 사용성 테스트로 임계값 도출

일반 채팅과 달리 AI 코칭 대화에서는, 사용자가 메시지를 나눠 보내는 도중에 AI가 끼어들어 응답하면 대화가 어색해집니다. 그래서 여러 메시지를 하나의 턴으로 묶어 보내는 정책이 필요했습니다.

○ 고민

"사용자의 입력이 끝났다"를 판단할 기준과 "전송이 실패했다"를 판단할 기준이 필요했는데, 참고할 만한 선례가 없었습니다.

● 결정

데모 앱으로 사용성 테스트를 직접 설계해 측정했고, 목적이 다른 두 타이머를 분리했습니다. 하나는 마지막 입력 후 일정 시간 입력이 없으면 턴이 끝났다고 보고 묶어 보내는 기준(2.5초)이고, 다른 하나는 보낸 뒤 응답을 기다리다 실패로 판단하는 기준(7초)입니다. 재는 대상도 목적도 다르기 때문에 하나의 값으로 합칠 수 없었습니다. 기준이 짧으면 사용자가 잠시 생각하는 사이 AI가 끼어들고, 길면 응답이 느리다고 느껴지기 때문에 그 양쪽의 균형점을 테스트로 찾았습니다.

FCM 푸시 알림 안정성 개선

코치가 보내는 푸시 알림은 사용자가 다시 앱으로 돌아와 코칭을 이어가게 만드는 핵심 동선입니다.

- **발견** 아침에 일어나 보니 와 있어야 할 코치 푸시가 오지 않았습니다. 원인을 추적해 보니, 알림에 넣을 썸네일을 불러오는 과정에서 주소 조회(DNS)가 실패하면 예외 처리(try-catch)가 없어 알림 생성 자체가 중단되고 있었습니다. 이미지 한 장 때문에 알림 전체를 놓치는 구조였습니다.
- **원인 분석** 어떤 조건에서 DNS 조회가 실패하는지 추적한 결과, 배터리 절약 모드에서 기기가 Doze 모드에 들어간 상태일 때 높은 확률로 재현되는 것을 확인했습니다. Doze의 백그라운드 네트워크 제한이 원인으로 추정되는데, 이는 OS 정책이라 앱에서 막을 수 없는 영역입니다. 그래서 네트워크가 제한돼도 알림이 뜨도록, 이미지 로딩의 네트워크 의존 자체를 없애는 방향으로 잡았습니다.
- **해결** 세 단계에 걸쳐 점진적으로 개선했습니다.
 - 1차**
썸네일 로딩을 try-catch로 감싸, 이미지를 못 불러와도 알림은 항상 뜨도록 했습니다.
 - 2차**
일시적인 네트워크 불안정에 대비해 이미지 로딩에 3회 재시도와 연결, 읽기 제한 시간을 적용했습니다.
 - 3차 (근본 개선)**
매번 네트워크로 이미지를 불러오는 구조 자체를 바꿨습니다. 앱을 켤 때 코치 이미지를 미리 로컬에 저장(캐싱)해 두고, 마지막으로 받아온 시점을 기록해 24시간에 한 번씩만 갱신하도록 했습니다(syncIfNeeded). 네트워크가 제한된 상태에서도 저장해 둔 이미지로 알림을 표시합니다.
- **결과** 발견 당시와 같은 조건(배터리 절약과 Doze 모드)을 재현해, 이미지를 포함한 알림이 정상적으로 표시되는 것을 확인했습니다.

경력 공백

2024.06 – 2025.05

Swit Korea의 구조조정으로 퇴사한 뒤, 다음 커리어를 신중히 정하면서 사이드 프로젝트로 Android 역량을 유지했습니다(파타타, WatchTimer는 아래 Other Projects 참고). 이후 Android 단독 개발 환경을 책임질 수 있는 회사 Tyrell에 합류했습니다.

협업 도구 Swit의 Developers 프로젝트에서 Android를 담당했습니다. Android, iOS, Web, Server, 기획, 디자인, 보안 등 총 10명이 참여한 프로젝트에서, 여러 진입 경로에 걸쳐 일관된 동작을 설계하는 경험을 했습니다.

딥링크 진입 구조 개선

- **발견** 댓글 화면에서 지원하지 않는 딥링크를 누르면, 먼저 메인 화면으로 넘어간 뒤에야 "지원하지 않는다"는 안내(Alert)가 뜨는 어색한 동작이 있었습니다. 원인은 MainActivity 하나가 모든 딥링크의 진입점이면서, 동시에 링크 검증과 화면 전환 책임까지 떠안고 있던 구조였습니다. 클릭 시점에 검증을 끼워 넣거나 설정(manifest)으로 우회하려 했지만 시스템 제약으로 막혔습니다.
- **해결** 진입점 자체를 분리했습니다. 모든 딥링크 처리를 화면이 보이지 않는 DeeplinkActivity로 옮겨 검증, 이동, 스스로 종료(finish) 순서로 단일화했고, 지원하지 않는 딥링크는 화면 전환 없이 안내만 띄우고 종료되도록 했습니다.

Featured Project

Took (특)

2026.04 – 현재

개인 프로젝트, Play Store 배포. 떠오른 순간 놓치기 쉬운 할 일을 자연어 한 줄로 빠르게 등록하는 리마인더 앱입니다. 기획부터 배포, 운영까지 혼자 진행했습니다. 현재 약 30여 명이 사용 중이며, 출시 직후부터 사용자 행동을 직접 관찰하며 개선해 왔습니다.

Kotlin

Jetpack Compose

AlarmManager

Cloud Firestore

데이터로 사용자를 이해하기: 행동 로그 설계와 개선

사용자가 동의했을 때만 입력 데이터를 로그로 변환해 Firebase에 저장하되, 단순 집계가 아니라 무엇이 문제인지 역추적할 수 있도록 네 가지로 분류했습니다.

- 정상적으로 등록된 일정
- 시간이 빠진 일정: 앱이 시간을 잘못 해석한 것인지, 사용자가 애초에 시간을 적지 않은 것인지 구분하기 위함
- 등록한 지 5초 안에 완료 처리된 일정: 원하는 시간에 일정이 잡히지 않아 사용자가 곧바로 완료시켜 치워버린 경우를 잡아내려는 가설

- 캘린더에서 매칭되는 일정을 찾지 못한 입력: 입력에 중요 키워드가 있는데도 캘린더에 연결할 일정이 없을 때

● **결과 ①** **타겟 가설 수정.** 직장인이 주 사용자일 거라 가정하고, 등교와 하교 같은 학생의 일상은 일부러 인식 패턴에 넣지 않았습니다. 그런데 '5초 안에 완료' 로그에 "하교 후"로 시작하는 입력이 잡혔습니다. 학생 일상을 패턴에 넣지 않은 탓에 시간이 제대로 잡히지 않았고, 사용자가 곧바로 완료 처리한 것이었습니다. 학생 수요가 실제로 있다는 것을 데이터로 확인하고, 등교와 하교 패턴을 추가하는 방향으로 개선하고 있습니다.

● **결과 ②** **문제를 사용자보다 먼저 발견.** '캘린더 매칭 실패' 로그에서 "병원" 입력이 캘린더와 연결되지 못한 것을 봤습니다. 단순히 캘린더에 해당 일정이 없어서라고 볼 수도 있었지만, 혹시 캘린더 연동 자체가 막힌 것은 아닌지 의심했습니다. 확인해 보니 Google Calendar API가 정식(프로덕션)이 아닌 테스트 모드로 설정돼 있었고, 현재 정식 전환 심사를 신청한 상태입니다. 이후 실제로 캘린더 연동이 안 된다는 사용자 문의가 들어왔는데, 로그 설계 덕분에 문의가 오기 전에 이미 원인을 파악하고 대응하고 있었습니다.

선택과 집중: 서버 없이 동작하는 무료 앱

행동 로그를 제외한 핵심 기능은 모두 기기 안(온디바이스)에서 동작합니다. 혼자 만들고 운영하는 앱이라 서버 운영비가 곧 서비스의 지속 가능성을 좌우하기에, 사용자에게 추가 비용을 지우지 않고 무료로 제공하는 것을 목표로 삼았습니다. 원래라면 서버가 필요했을 기능도 온디바이스에서 처리하는 방법을 찾아 서버 없이 동작하도록 설계했고, 그 결과 운영비가 들지 않아 지속적으로 무료 제공이 가능한 구조로 배포했습니다.

Other Projects

파타타

2025.01 – 2025.03

Kotlin

MVVM

Clean Architecture

Hilt

Flow

Coroutine

Naver Map SDK

- 이미지 크기와 화질의 균형점을 실험으로 찾아, 한 장당 30MB이던 이미지를 4MB로 줄이고 로딩 시간을 3초에서 1초로 단축했습니다.
- Play Store 배포 후에만 나타난 Google 로그인 실패를 추적했습니다. Play Store가 앱에 자체 서명 키를 다시 발급한다는 점을 파악하고, 그 키의 SHA-1 값을 GCP에 추가 등록해 해결했습니다. 로컬에서는 재현되지 않고 배포 환경에서만 발생하는 이슈를 끝까지 추적한 경험입니다.

Kotlin

Compose

MVI

Coroutine

Service

Vibrator API

- 갤럭시 워치의 절전 화면(Ambient Mode)에서 백그라운드 동작(Service)이 멈추는 문제를 만났습니다. 공식 예제, 픽셀 워치 에뮬레이터와 비교해 갤럭시 워치 4의 기기 자체 제약임을 확인하고, 해당 API에 의존하는 대신 사용자 동작을 기준으로 시간을 추적하는 자체 상태 머신으로 우회했습니다.

Other Experience

GDSC DJU · Core(운영진) & Client Position Lead

2022

- 30명 규모의 GDSC DJU에서 운영진으로 활동했고, Elice의 협찬을 받아 코딩 테스트 대회를 주최하고 운영했습니다.

Compose Camp · Instructor

2022

- Google이 주최한 Compose Camp에서 550명이 신청한 입문 코스 Unit 1 교육을 맡았습니다. 코스 진행 중 질문 답변과 문의 처리, 함께 코딩하는 이벤트 기획을 담당했습니다.

Education

대진대학교 · 컴퓨터공학 전공

2017.03 – 2023.08

학점 3.9 / 4.5